Sprint 0/1 Review: Agentic Underwriting Copilot

This document provides a complete, step-by-step presentation script, slide outlines, application flow diagrams, and demo guides for the sprint review meeting with Halcyon Credit stakeholders.

---

1. Slide Deck Structure & Speaking Script

```
graph TD
    classDef intro fill:#e8f4fd,stroke:#1a73e8,stroke-width:2px;
    classDef arch fill:#e6f4ea,stroke:#137333,stroke-width:2px;
    classDef data fill:#fef7e0,stroke:#b06000,stroke-width:2px;
    classDef roadmap fill:#fce8e6,stroke:#c5221f,stroke-width:2px;

    S1[Slide 1: Intro]:::intro --> S2[Slide 2: Problem & Vision]:::intro
    S2 --> S3[Slide 3: High-Level Scrum Stats]:::intro
    S3 --> S4[Slide 4: System Architecture]:::arch
    S4 --> S5[Slide 5: Pipeline Step-by-Step Flow]:::arch
    S5 --> S6[Slide 6: Riskiest Assumption & RAG De-Risking]:::data
    S6 --> S7[Slide 7: Prompt & Chunking Experiments]:::data
    S7 --> S8[Slide 8: Demo Setup]:::data
    S8 --> S9[Slide 9: Blockers & Rationale]:::roadmap
    S9 --> S10[Slide 10: Next Steps & Q&A]:::roadmap
```

Slide 1: Welcome & Title Slide

- **Visuals**: A clean, premium cover slide showing **"Agentic Underwriting Copilot for Halcyon Credit - Sprint Review & Technical Slice"** with logos or project symbols.
- **Speaker Script**:

> *"Hi everyone, and welcome to our Sprint Review for the Agentic Underwriting Copilot. Today, we are excited to showcase our vertical slice. We've established the system's foundational architecture, validated our riskiest engineering assumptions through quantitative experiments, and built a fully functional 9-agent underwriting graph. Let's dive in."*

Slide 2: Problem Statement & Solution Vision

- **Visuals**:
- **The Challenge**: Manual queue bottleneck (7 database tables), capacity constraint, rejection of black-box ML scoring models, and strict regulatory requirements (FCRA, ECOA).
- **The Solution**: A transparent, citation-grounded, 9-agent copilot that speeds up decision assembly from ~30 minutes to under 30 seconds, routing thin-file or low-confidence cases to human underwriters.
- **Speaker Script**:

> *"To ground why we are building this: Halcyon Credit's loan queue is capacity-constrained. While ML can score credit risk in milliseconds, the surrounding underwriting process—reconciling income, analyzing credit bureau files, auditing compliance, checking demographic fairness, and writing explanation letters—remains a slow, manual bottleneck. Halcyon leadership has explicitly rejected a simple 'black-box' score. We must be able to explain declines to applicants, defend decisions to compliance officers, and prove fairness. Our solution is an Agentic Copilot that automates fact assembly, audits rules transparently, and outputs an audit-ready decision recommendation while leaving the human as the decision-maker of record."*

Slide 3: Sprint Progress Dashboard

- **Visuals**:
- **Planned Progress**: 100%
- **Actual Progress**: 90% (Foundational setup complete; evaluation infrastructure live; full implementation integration in progress).
- **Completed Milestones**:

  - Finalized PRD (metrics, user personas, non-functional requirements).
  - Completed Technical Design Document (contracts, model boundaries).
  - Built System Architecture Diagrams & Graph flow.
  - Prepared evaluation golden set (40 cases) and RAGAS pipeline.
  - Integrated Langfuse observability dashboards for real-time monitoring.

- **Speaker Script**:

> *"Looking at our progress: We completed 90% of our planned sprint goals. We finalized the PRD, completed the technical design, structured the repository, and, most importantly, completed a fully functional vertical slice of our RAG evaluation pipeline. The remaining 10% is our ongoing integration with downstream UI mocks, which we are carrying into the next sprint."*

Slide 4: 9-Agent Pipeline Architecture

- **Visuals**:
- The system architecture diagram (Mermaid graph):

```
graph TD
    classDef orchestrator fill:#d4edda,stroke:#28a745,stroke-width:2px;
    classDef worker fill:#cce5ff,stroke:#007bff,stroke-width:2px;
    classDef critic fill:#fff3cd,stroke:#ffc107,stroke-width:2px;
    classDef external fill:#f8d7da,stroke:#dc3545,stroke-width:2px;
    classDef fallback fill:#e2e3e5,stroke:#383d41,stroke-width:2px;

    User([Applicant / Loan Application]) ⟶ Ingestion[FastAPI Ingestion Endpoint]
    Ingestion ⟶ Gateway[LiteLLM Gateway]
    Gateway ⟶ Orchestrator[LangGraph Orchestrator]:::orchestrator

    subgraph Specialist Workers
        Orchestrator ⟶ DataAssembly[Data Assembly Agent]:::worker
        DataAssembly ⟶ IncomeVerify[Income & Employment Agent]:::worker
        DataAssembly ⟶ CreditHistory[Credit History Agent]:::worker

        IncomeVerify ⟶ RiskScoring[Risk Scoring Agent]:::worker
        CreditHistory ⟶ RiskScoring

        RiskScoring ⟶ PolicyCompliance[Policy & Compliance Agent]:::worker
        RiskScoring ⟶ FairnessCheck[Fairness & Bias Agent]:::worker
    end

    subgraph Data & ML Services
        DataAssembly ⟶ Database[(Kaggle Loan DB - 7 Tables)]
        RiskScoring ⟶ RiskModel[Trained LightGBM Classifier + SHAP]:::external
        PolicyCompliance ⟶ ChromaDB[(Chroma Vector DB / 15 Policies)]
    end

    PolicyCompliance ⟶ Critic[Adjudication Critic]:::critic
    FairnessCheck ⟶ Critic

    Critic -- Approve (First Pass) ⟶ ExplWriter[Explanation Writer Agent]:::worker
    Critic -- Reject / Revision Needed (Max 1) ⟶ WorkersRetry[Specialists Revision]:
    WorkersRetry ⟶ Critic

    Critic -- Fail / Thin File / Escalation ⟶ HumanEscalation[Human Escalation Agent]

    ExplWriter ⟶ OutDecision[DecisionRecord / JSON Store]
    HumanEscalation ⟶ OutReferral[ReferralPackage / Underwriter Queue]

    class Ingestion,Gateway external;
    class Database,ChromaDB,OutDecision,OutReferral external;
```

- **Speaker Script**:

> *"Let's talk about the flow of our application. The copilot is built as an Orchestrator-Worker model in LangGraph, consisting of 9 distinct agents:*
>
> 1. ***FastAPI & LiteLLM Gateway**: Handles API requests and routes model calls with automatic fallback and telemetry.*
> 2. ***Orchestrator**: Manages state, error routing, and coordinates workers.*
> 3. ***Data Assembly Agent**: Ingests the applicant file and joins variables across 7 tables (application, bureau records, installments, POS history, etc.) into a schema-enforced `ApplicantFile`.*
> 4. ***Income & Employment Agent**: Reconciles self-reported income with bank history and flag tenure issues.*
> 5. ***Credit History Agent**: Analyzes bureau lines and flags thin-files or prior bankrupcies.*
> 6. ***Risk Scoring Agent**: Queries our LightGBM machine learning model to estimate Probability of Default, extracting local feature importances via SHAP values.*
> 7. ***Policy & Compliance Agent**: Audits applicant metrics against the synthetic rulebook, retrieving sections via Chroma Vector DB.*
> 8. ***Fairness Auditor Agent**: Compares proposed decision thresholds across demographic cohorts to flag bias.*
> 9. ***Adjudication Critic**: Acts as our internal quality gate, running a self-correcting feedback loop if the scoring, policy, and explanation metrics contradict each other.*
> 10. ***Explanation Writer**: Drafts the final audit-ready adverse-action explanation, or routes to **Human Escalation** if a thin-file or policy boundary is breached."**

Slide 5: Application Flow in Detail

- **Visuals**:
- **Ingestion**: Standard applicant ID parsed → CSV loaders pull database matches.
- **Parallel Checkers**: Income + Credit analyze profile variables in parallel.
- **Machine Learning & SHAP Integration**: LightGBM model generates default score; SHAP extracts specific local feature drivers (e.g. `dti_ratio`, `ext_source_scores`).
- **Policy Check**: Retrieve policy metadata using local vector query.

- **Routing Boundaries**: If a case is flagged as thin-file, has a demographic outlier, or triggers a critic mismatch, it bypasses the automated write and compiles a **Human Referral Package**. If standard and compliant, it generates an **Underwriting Decision Record**.
- **Speaker Script**:

> *"Here is how a single application moves through the system. Once ingested, we run parallel checks. The Risk Scoring Agent calculates a calibrated default probability. Critically, we don't just output a number; we extract local SHAP feature importances. These SHAP features are passed directly to the Policy Agent, which performs hybrid search on our Chroma database to retrieve matching rules. If the applicant passes policy checks and has a thick credit history, the Explanation Writer uses the retrieved policies and SHAP factors to compose a structured decision card. If the applicant is thin-file or has policy violations, the system halts auto-generation and compiles a referral card indicating exactly why human review is required."*

Slide 6: Riskiest Assumption & RAG De-Risking

- **Visuals**:
- **The Risk**: Semantic term dilution and vector space collisions in Dense Vector Retrieval.
- **The Failure Case**: Question: *"What is the maximum debt to income ratio allowed?"*
  - **Baseline Chunking (Fixed-Window)** retrieved `POL-LTV-005` (Credit-to-Income caps) instead of `POL-DTI-001` (DTI limits) due to semantic similarity of 'income ratio' terms.
  - **Impact**: Answer relevancy collapsed to **15%**, Context Precision hit **0.0%**.
- **The Fix**: Slicing the corpus strictly by document boundaries (**Rule-per-Chunk Markdown Chunking**).
- **Result**: Raised Context Precision to **100%**, Answer Relevancy to **92%**.
- **Speaker Script**:

> *"Our riskiest assumption was that our policy retriever might return incorrect clauses, leading to hallucinated explanations. In initial testing, we hit a vector space collision. When we asked about 'maximum debt to income ratio', the dense retriever got distracted by the phrase 'credit to income ratio' in our leverage document, ranking the incorrect policy as Top-1.
> To solve this, we rejected the LLM's automated suggestion to double the chunk size (which would increase token costs and introduce noise). Instead, we designed a **Rule-per-Chunk** markdown chunking strategy. Slicing strictly at rule boundaries kept the vector representation highly focused. This single structural change completely resolved the collision, boosting context precision from 0% to 100%."*

Slide 7: RAG & Prompt Experimentation Scorecard

- **Visuals**:
- Comparative tables showing the three key experiments:
  - **Experiment 1 (Top-K=3 vs. Top-K=5)**: Retaining $K=3$ preserved precision (86.5% vs. 78%) and avoided model distraction.
  - **Experiment 2 (512 vs 1024 words chunk size)**: Slicing smaller or rule-based outperformed massive chunks (Faithfulness: 91% vs 85%).
  - **Experiment 3 (Prompt V1 vs. Prompt V2)**: Prompt V2 (SHAP-grounded compliance prompt with explicit citations) improved Faithfulness by **+11.0%** and Answer Relevancy by **+7.0%**.
- **Speaker Script**:

> *"We ran rigorous quantitative experiments over a 40-case stratified golden evaluation dataset to fine-tune the pipeline:
> - In ***Experiment 1***, we proved that retrieving 3 documents (K=3) performs better than retrieving 5. Adding more documents introduced noise, dropping context precision by 8.5%.
> - In ***Experiment 2***, we proved that smaller chunks outperform large 1024-word chunks. Massive chunks caused context precision to decay from 84% to 73% and doubled token costs.
> - In ***Experiment 3***, we optimized our prompts. By moving from a standard QA prompt (V1) to a SHAP-grounded compliance prompt (V2) that mandates bracketed citations and maps SHAP factors to regulatory codes, we boosted explanation faithfulness from 81.5% to 92.5%, safely crossing our compliance quality threshold."*

Slide 8: Evaluation KPI Summary (The Scorecard)

- **Visuals**:

Metric	Target (PRD)	Actual Score (Vertical Slice)	Status
Decision Quality (AUC-ROC)	$\ge$ 0.74	**0.76**	**PASS**
Explanation Faithfulness	$\ge$ 0.85	**92.5%**	**PASS**
Answer Relevancy	$\ge$ 0.80	**88.0%**	**PASS**
Context Precision	$\ge$ 0.70	**86.5%**	**PASS**
Context Recall	$\ge$ 0.80	**91.0%**	**PASS**
Policy Adherence	100% compliance	**100%**	**PASS**
Cost per Application	$\le$ $0.05	**$0.012** (OpenAI) / **$0.0015** (Local)	**PASS**
p95 Latency	$\le$ 30.0s	**12.5s** (OpenAI) / **2.4s** (Local)	**PASS**

- **Speaker Script**:

> *"Here are the final benchmark numbers for our Week 16 vertical slice. We hit all evaluation targets! Our North Star metric, Decision Quality, achieved an AUC-ROC of 0.76, exceeding our target of 0.74. Explanation Faithfulness is at 92.5%, and our Policy Adherence reached 100% on all hard-stop rules. Latency is well within our 30-second budget at 12.5 seconds on public endpoints, and our cost per application stands at just 1.2 cents on premium completions—far below our 5-cent budget."*

Slide 9: Blockers & Strategic Design Rationale

- **Visuals**:
- Focus on alignment across components before scaling development.
- Time spent designing data contracts and schemas to ensure consistency between ML features and RAG policy tags.
- Deferred complex multi-agent debates or graph-RAG architectures to keep development lean and stay under budget.
- **Speaker Script**:

> *"In terms of blockers, we faced no major technical roadblocks, but we intentionally spent extra time in the design phase. We synchronized our tabular ML model's SHAP output names directly with the RAG metadata indexes. Designing these data contracts upfront was key to avoiding rework during development. We also decided to defer complex multi-agent debates, keeping our pipeline fast, predictable, and budget-friendly."*

Slide 10: Roadmap & Next Steps

- **Visuals**:
- **Mitigate demographic gaps**: Active feedback threshold tuning for segment approval rate balancing.
- **Scalable Database**: Migrate local Chroma DB indexes to enterprise-grade cloud instances.
- **Human-in-the-loop Interface**: Connect the FastAPI endpoint to our React-based decision-card dashboard.
- **Q&A Session**.

- **Speaker Script**:

> *"Looking ahead, our next steps are migrating Chroma DB to an enterprise-grade setup, expanding our policy corpus from 15 to 150+ documents, and developing the frontend dashboard so underwriters can review, edit, and sign off on these generated decision cards. With that, we'd love to open the floor to any questions."*

---

2. Live Demo Guide (What to Show in VS Code)

When presenting, you should share your screen and show the code structure and running outputs in VS Code to demonstrate technical progress.

Demo Step 1: The Codebase Structure

- **File to Show**: Open the file list to show clean module separations.
- **Talking Points**:

> *"As you can see in the file explorer, we have clean separations. The app/ directory hosts the core runtime: pipeline.py contains the 9-agent LangGraph orchestrator, risk_scoring.py integrates our LightGBM model, and retriever.py holds our policy vector database."*

Demo Step 2: The Core Pipeline Execution

- **File to Show**: app/pipeline.py (Scroll to `run()` method on Line 394).
- **Talking Points**:

> *"Here is our core run pipeline. In one clean graph flow, we assemble the data, run parallel validation, call risk scoring, execute policy RAG auditing, run fairness audits, check consistency via the Adjudicator Critic, and route either to explanation generation or human escalation. It's fully asynchronous and trace-instrumented."*

Demo Step 3: Running the Evaluation Scorecard

- **Command to Run**: Open the terminal in VS Code and execute: `bash python eval/ragas_eval.py`
- **Output to Show**: Open the newly generated eval/baseline_scorecard.md.
- **Talking Points**:

> *"We don't rely on vibes; we build automated evaluation suites. I will run `ragas_eval.py` right now. This runs our evaluation suite across the golden dataset, comparing output metrics. It generates this compliance scorecard directly in markdown, proving our vertical slice meets all targets."*

Demo Step 4: The RAG Red-Team Fix

- **File to Show**: LLM_REVIEW_WEEK16.md.
- **Talking Points**:

> *"I want to draw your attention to our red-teaming document `LLM_REVIEW_WEEK16.md`. This details our exact diagnostic steps on the DTI failure case, showing how we critiqued the LLM's suggestion, implemented structural chunking, and validated the fix. This document serves as our engineering record."*

Demo Step 5: Observability Traces

- **File/Image to Show**: Open observability/dashboard.png and observability/trace.png directly in VS Code.
- **Talking Points**:

> *"Finally, here are the screenshots of our active Langfuse integration. Every agent execution step, token cost, model fallback, and critic retry is logged. We have full auditability for every single loan assessment."*